Elsevier DS Interview

RAG — Retrieval Augmented Generation

Detailed Q&A Revision Guide

Nityaa Kalra | August 2026

SECTION 1: RAG ARCHITECTURE OVERVIEW

Q1: What is RAG and why is it needed?

RAG (Retrieval Augmented Generation) grounds LLM responses in real, up-to-date documents instead of relying solely on parametric knowledge baked in during pretraining.

Problems RAG solves:

- LLMs hallucinate — they generate confident but wrong facts

- LLM knowledge has a cutoff date — can't answer about recent papers

- LLMs can't cite sources — RAG provides traceable provenance

- Fine-tuning is expensive — RAG updates knowledge by updating the index, not the model

At Elsevier: RAG over Scopus or ScienceDirect lets a model answer questions about scientific literature without hallucinating references or facts.

Q2: Walk me through the end-to-end RAG pipeline.

There are two phases: indexing (offline) and querying (online).

Indexing phase (offline):

1. Load documents (PDFs, HTML, text)

2. Chunk: split into smaller pieces (e.g., 512 tokens with 50-token overlap)

3. Embed: encode each chunk with an embedding model into a dense vector

4. Store: save vectors in a vector database (Qdrant, Chroma, Pinecone, FAISS)

Query phase (online):

1. Take user query

2. Retrieve: find top-k similar chunks from vector DB + optionally BM25

3. Optionally rerank: cross-encoder reranker to reorder by relevance

4. Augment: build prompt with retrieved context + original query

5. Generate: LLM produces answer grounded in the retrieved context

Q3: What is chunking and what strategies exist?

Chunking splits documents into retrievable pieces. Chunk size is a critical hyperparameter — too small loses context, too large adds noise.

Strategy	How it works	Best for
Fixed size	Split every N tokens, no regard for content	Simple baseline
Sentence	Split at sentence boundaries	General text
Paragraph	Split at double newlines	Documents with structure
Semantic	Split when meaning shifts (embedding similarity drop)	Academic papers
Recursive	Split at \n\n, then \n, then space, until chunk small enough	LangChain default

Overlap: chunks should overlap by ~10-20% so relevant info at chunk boundaries isn't split across two chunks and missed.

SECTION 2: RETRIEVAL

Q4: What is dense retrieval vs sparse retrieval?

Dense retrieval: encodes query and documents as continuous vectors using a neural embedding model. Finds semantically similar documents even if they use different words. Example: 'climate change' matches 'global warming'.

Sparse retrieval (BM25): keyword-based. Scores documents by term frequency (TF) and inverse document frequency (IDF). Fast, no GPU needed, exact keyword matching.

	Dense	Sparse (BM25)
Semantic understanding	Yes	No
Exact keyword match	Poor	Excellent
Requires GPU	Yes	No
Speed	Slower	Very fast
Out-of-vocabulary words	Handles well	Fails

Q5: What is BM25 and how does it score documents?

BM25 (Best Match 25) is the gold standard sparse retrieval algorithm. It extends TF-IDF with document length normalization and saturation (diminishing returns for term frequency).

The score for a document D given query Q is:

```
Score(D, Q) = sum over query terms t of: IDF(t) * (tf(t,D) * (k1+1)) / (tf(t,D) + k1*(1 - b + b*|D|/avgdl)) Where: tf(t,D) = term frequency of t in document D IDF(t) = log((N - df + 0.5) / (df + 0.5) + 1) [inverse doc freq] |D| = document length in words avgdl = average document length in corpus k1 = 1.5 (term frequency saturation) b = 0.75 (length normalization strength)
```

Key insight: terms that appear in few documents (high IDF) and appear multiple times in this document (high TF) get the highest scores.

Q6: What is Hybrid Retrieval and RRF?

Hybrid retrieval combines dense and sparse rankings. Neither is universally better — hybrid catches what each misses.

RRF (Reciprocal Rank Fusion): merges ranked lists from multiple retrievers without needing to normalize their scores. Formula:

```
RRF_score(doc) = sum over each ranker r of: 1 / (k + rank_r(doc)) Where k = 60 (smoothing constant, prevents top ranks dominating too much) Example: document ranked 1st by dense and 3rd by BM25: RRF = 1/(60+1) + 1/(60+3) = 0.01639 + 0.01587 = 0.03226 Document ranked 5th by dense and 2nd by BM25: RRF = 1/(60+5) + 1/(60+2) = 0.01538 + 0.01613 = 0.03151 First document wins — consistently top across both retrievers is strongest signal
```

RRF is used in your project's hybrid retrieval setup. Know this well.

Q7: What is a reranker and how does it differ from retrieval?

Retrieval (bi-encoder): encodes query and document separately, computes dot product. Fast — can search millions of docs. But less accurate.

Reranker (cross-encoder): takes query + document together as input, outputs a single relevance score. Much more accurate (sees interaction between query and document words) but too slow for full corpus. Used to reorder the top 50-100 retrieved candidates.

```
from sentence_transformers import CrossEncoder reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2') # Input: list of (query, document) pairs pairs = [(query, doc) for doc in retrieved_docs] scores = reranker.predict(pairs) # Sort by reranker score reranked = sorted(zip(retrieved_docs, scores), key=lambda x: x[1], reverse=True) top_docs = [doc for doc, _ in reranked[:5]]
```

SECTION 3: EMBEDDINGS & VECTOR DATABASES

Q8: What is an embedding and how are they used in RAG?

An embedding is a dense vector (e.g., 384 or 768 dimensions) representing the semantic meaning of text. Similar texts have vectors that are close together in the embedding space, measured by cosine similarity.

```
from sentence_transformers import SentenceTransformer model =
SentenceTransformer('all-MiniLM-L6-v2') # your config model # Embed documents chunks
= ['chunk 1 text...', 'chunk 2 text...'] embeddings = model.encode(chunks) # shape:
[n_chunks, 384] # Embed query query_vec = model.encode('What is stance detection?') #
shape: [384] # Cosine similarity from sklearn.metrics.pairwise import
cosine_similarity scores = cosine_similarity([query_vec], embeddings) # [1, n_chunks]
```

Q9: What is a vector database and how does FAISS/Qdrant work?

A vector database stores embeddings and supports fast approximate nearest neighbor (ANN) search — finding the k most similar vectors to a query vector without comparing against every stored vector.

Common options:

- **FAISS**: Facebook's library, runs in-memory or on disk, excellent performance, no server

- **Qdrant**: open-source vector DB server, supports hybrid search, filtering, payload storage. You use Qdrant Cloud.

- **Chroma**: lightweight, easy to set up, good for prototyping

- **Pinecone**: fully managed cloud, no self-hosting

```
from qdrant_client import QdrantClient from qdrant_client.models import Distance,
VectorParams, PointStruct client =
QdrantClient(url='https://your-cluster.qdrant.io', api_key='...') # Create
collection client.create_collection('scientific_docs',
vectors_config=VectorParams(size=384, distance=Distance.COSINE)) # Index documents
client.upsert('scientific_docs', points=[ PointStruct(id=i, vector=emb.tolist(),
payload={'text': chunk}) for i, (chunk, emb) in enumerate(zip(chunks, embeddings)) ])
# Query results = client.search('scientific_docs', query_vector=query_vec.tolist(),
limit=5) for r in results: print(r.payload['text'], r.score)
```

SECTION 4: RAGAS EVALUATION

Q10: How do you evaluate a RAG system? What is RAGAS?

RAGAS is a framework for evaluating RAG pipelines without requiring human-labeled reference answers (reference-free). It uses an LLM as judge.

Metric	What it measures	Range
Faithfulness	Are claims in the answer supported by the retrieved context?	0-1
Answer Relevance	Does the answer actually answer the question asked?	0-1
Context Precision	Are retrieved chunks actually relevant (no noise)?	0-1
Context Recall	Did retrieval find all relevant chunks needed to answer?	0-1

Faithfulness is the most important — it directly measures hallucination. If faithfulness is low, the model is adding information not in the retrieved context.

```
from ragas import evaluate from ragas.metrics import faithfulness, answer_relevancy, context_precision, context_recall dataset = { 'question': ['What is stance detection?'], 'answer': ['Stance detection classifies opinions...'], 'contexts': [['chunk 1', 'chunk 2']], 'ground_truths': [['Reference answer if available']] } result = evaluate( dataset, metrics=[faithfulness, answer_relevancy, context_precision, context_recall] ) print(result) # dict with scores per metric
```

Q11: When should you use RAG vs fine-tuning vs both?

Scenario	Best approach
New knowledge not in model training data	RAG
Need to cite specific documents/sources	RAG
Improve tone, format, task style	Fine-tuning
Model too generic for specialized domain	Fine-tuning
Up-to-date knowledge + task-specific behavior	RAG + Fine-tuning

Your stance detection project: you use both. SLMs evaluated zero-shot, fine-tuning in progress. A natural next step you should mention: add RAG over domain-specific stance corpora to improve context grounding.